

Studio OS - Manuel humain

Table of Contents

Vision produit - Studio OS

Resume

Studio OS est le systeme d'exploitation interne du studio. Il ne remplace pas Godot, Git, Claude, Qwen, Obsidian ou Graphify: il les relie afin que deux developpeurs et leurs agents IA partagent le meme contexte operationnel.

Probleme resolu

Sans Studio OS, l'information est fragmentee: une tache est dans un outil, une decision dans une discussion, un commit sur GitHub, une note dans Obsidian, un graphe dans Graphify, une capture video sur un disque local et une IA ignore souvent ce que l'autre humain ou l'autre agent est en train de faire.

Studio OS centralise les liens entre ces objets sans obliger a centraliser tous les fichiers de travail.

Proposition de valeur

Studio OS doit permettre de repondre immediatement a ces questions: - Sur quoi travaille chaque humain ? - Quels agents travaillent actuellement ? - Quelles taches sont actives, bloquees ou en review ? - Quels fichiers/sous-systemes risquent un conflit ? - Quelles decisions architecturales s'appliquent a cette tache ? - Quel contexte faut-il donner a Claude ou Qwen ? - Quels travaux IA attendent une validation ? - Quel build est le dernier valide ? - Quels moments de capture peuvent alimenter le marketing ? - Quels fichiers mon collegue m'a envoyes et pourquoi ?

Architecture humaine

Le studio comporte deux developpeurs a distance. Chacun garde ses outils locaux. Un VPS OVH central heberge les services partages: API, MCP, base PostgreSQL, dashboard, event store et stockage objet.

Ce que Studio OS n'est pas

- Ce n'est pas un IDE.
- Ce n'est pas un moteur de jeu.
- Ce n'est pas un remplacement de Git.
- Ce n'est pas un outil de surveillance de productivite.

- Ce n'est pas un systeme de partage de disque reseau.
- Ce n'est pas une IA unique qui controle tout.

Philosophie

L'outil doit automatiser la documentation du travail plutot que forcer les developpeurs a remplir manuellement des formulaires. L'etat doit etre derive autant que possible de Git, des sessions de travail, des agents et des integrations locales. # Fonctionnalites finales prevues

Coordination du studio

- Tableau de bord global des projets, utilisateurs, machines et agents.
- Presence en ligne/idle/offline via heartbeat.
- Taches avec proprietaire humain ou agent, priorite, statut et dependances.
- Sessions de travail liees a une tache, une branche, une machine et des agents.
- Timeline des actions importantes.

Collaboration et conflits

- Soft locks sur fichier, dossier, scene, ressource, module ou sous-systeme.
- Detection des claims qui se chevauchent.
- Detection des branches qui modifient les memes fichiers.
- Alertes de dependances bloquees.
- Suggestions d'action sans blocage de Git.

IA et orchestration

- MCP central pour Claude Code.
- Studio Producer qui analyse priorites, blocages et parallelisation.
- Decomposition d'une grosse tache en sous-taches avec dependances.
- Recommandation d'executant: humain, Claude ou Qwen.
- AI Work Ledger pour tracer requete, resultat, fichiers, tests et review.
- Review Queue commune pour code IA, PR, decisions, memoire, builds et conflits.

Memoire et connaissance

- Memoire privee locale, memoire projet partagee, memoire studio partagee.
- Recherche dans Obsidian depuis les agents autorises.
- Propositions de memoire avec approbation humaine.
- Qwen en lecture seule sur la memoire partagee par default.
- Decision Log avec identifiants DEC-XXXX.

Graphify et contexte

- Graphify reste local.
- Requete de dependances, symboles et fichiers pertinents.
- Generation de Context Packages par tache.
- Regroupement de la tache, decisions, commits, graphe, memoire, dependances et etat du projet.

Git, GitHub et builds

- Detection locale de branche, commit, statut et fichiers modifies.
- Association tache <-> branche <-> commit <-> PR <-> build.
- Integration GitHub pour PR, issues, commits et Actions.
- Statut de builds visible dans le dashboard.

Godot et enregistrements

- Detection du lancement/arret de Godot.
- Association automatique d'une session a un projet.
- Integration au systeme de capture existant.
- Marqueurs video manuels: studio mark "...".
- Generation de candidats marketing a partir de moments interessants.

Studio Transfer / Storage

- Transfert de fichiers entre les deux developpeurs.
- Upload et download resumables pour fichiers volumineux.
- Hash SHA-256 pour integrite.
- Historique, expiration et suppression.
- Association d'un transfert a un projet ou une tache.
- Categories: transfer, build, asset, recording, export, archive, other.
- Stockage via MinIO/S3 sur le VPS.
- URLs pre-signees temporaires.
- Drag & drop dans le dashboard et commandes CLI.
- Quotas et politique de retention.
- Les captures brutes restent locales par default.

Fonctionnement hors ligne

- SQLite local pour les evenements en attente.
- Retry avec backoff exponentiel.
- UUID et idempotency keys.
- Deduplication serveur.
- Resynchronisation automatique au retour de la connexion.

Notifications

Notifications uniquement pour les evenements qui demandent une action: conflit, build casse, review IA, PR prete, tache bloquee, decision/memoire a approuver, nouveau transfert. #

Workflows quotidiens

Demarrer une tache

1. L'utilisateur execute studio work APO-142 ou choisit la tache dans le dashboard.
2. Le daemon detecte projet, branche Git et etat Godot.
3. Une WorkSession est creee.

4. Le heartbeat publie l'activite.
5. Le systeme peut proposer les fichiers a claim via Graphify.
6. L'enregistrement de session peut demarrer automatiquement.
7. Claude charge le contexte via MCP.

Deleguer a Qwen

1. Claude ou le Producer identifie une sous-tache mecanique.
2. Une sous-tache est creee et attribuee a Qwen.
3. Qwen lit le contexte autorise.
4. Il travaille localement et journalise ses actions.
5. Le resultat passe en needs_review.
6. Un humain ou Claude valide avant fusion.

Prendre une decision d'architecture

1. Un humain/Claude cree une proposition.
2. La proposition indique contexte, choix, raisons et consequences.
3. Apres validation, Studio OS genere DEC-XXXX.
4. La decision devient recherchable par les futurs agents.

Eviter un conflit

1. Une tache claim scripts/player/player.gd.
2. L'autre developpeur ouvre une tache touchant le meme fichier.
3. Studio OS avertit du chevauchement.
4. Le developpeur peut continuer, changer de tache ou coordonner.

Envoyer un gros fichier

1. studio transfer send build.zip --to dev-b --project apotheosis.
2. L'API cree un Transfer et une URL d'upload pre-signee.
3. Le client envoie directement le fichier a MinIO en multipart.
4. A la fin, le hash est verifie et le transfert passe a ready.
5. Le destinataire recoit une notification.
6. Le download utilise une URL pre-signee et peut reprendre apres coupure.

Marquer un moment marketing

1. Pendant une capture: studio mark "first successful gameplay".
2. Le marqueur est lie a la session/tache/recording.
3. Le worker MarketingCandidate cree une proposition de clip.
4. Le pipeline marketing peut ensuite traiter uniquement cette plage temporelle.

Fin de journee

Studio OS genere un resume: taches, commits, travaux IA, decisions, builds, conflits, clips et transferts importants. # Deploiement sur VPS OVH

Services recommandés

- Caddy: reverse proxy et TLS.
- studio-api: FastAPI.
- studio-mcp: serveur MCP.
- PostgreSQL: données structurées.
- MinIO: stockage objet S3 compatible.
- studio-dashboard: interface web.
- workers: nettoyage, resume quotidien, notifications, synchronisations.

Schema

Internet -> Caddy -> API / MCP / Dashboard / MinIO endpoints signés. PostgreSQL et les ports d'administration ne sont jamais exposés publiquement.

Capacité initiale

Pour deux développeurs, un petit VPS est suffisant pour l'orchestration. Le stockage doit être dimensionné selon les builds/assets. Les vidéos brutes ne doivent pas être uploadées automatiquement.

DNS suggère

- studio.example.com -> dashboard/API
- mcp.example.com -> MCP si séparation utile
- storage.example.com -> MinIO/S3

Sauvegardes

- dump PostgreSQL quotidien.
- sauvegarde des secrets hors du VPS.
- copie/replication des objets importants selon budget.
- test de restauration documenté.

Mise à jour

- images Docker versionnées.
- migrations Alembic avant démarrage de la nouvelle API.
- rollback documenté.
- environnement staging recommande avant mises à jour structurantes. # Sécurité et exploitation

Règles essentielles

- HTTPS obligatoire hors localhost.
- Un token par machine, révocable indépendamment.
- Aucun secret dans Git.
- PostgreSQL non public.
- Buckets MinIO privés.

- URLs pre-signees courtes pour upload/download.
- Validation stricte des chemins/objets.
- Logs d'audit pour actions sensibles.

Permissions

Roles minimum: admin, developer, agent, readonly. Les agents ne doivent obtenir que les droits necessaires. Qwen est read-only sur la memoire partagee par default.

Transferts

- Limites de taille configurables.
- MIME et nom de fichier traites comme non fiables.
- Hash SHA-256.
- Antivirus optionnel si des fichiers externes entrent dans le systeme.
- Nettoyage automatique des transferts expires.

Confidentialite

Les chemins locaux, notes privees, captures brutes et caches Graphify ne sont pas publies par default. Les donnees partagees doivent etre explicitement classees projet/studio.

Observabilite

- logs structures.
- health checks.
- metriques CPU/RAM/disque.
- alertes si disque MinIO faible ou PostgreSQL indisponible. # Roadmap de construction

Phase 0 - Contrats

Figer API_CONTRACT, EVENT_CONTRACT, AUTH_SYNC_CONTRACT et schemas partages. Creer mocks.

Phase 1 - Core utilisable

PostgreSQL, auth machines, projects/tasks, sessions, heartbeat, claims, conflicts, events, MCP minimal, dashboard minimal.

Phase 2 - Client local

Daemon, SQLite offline queue, CLI, Git/Godot watchers, reconnect.

Phase 3 - Connaissance IA

AI Work Ledger, Review Queue, decisions, Obsidian/Graphify adapters, Context Packages.

Phase 4 - Storage

MinIO, TransferService, multipart, resumable download, UI/CLI, quotas, retention.

Phase 5 - Producer & automatisations

Producer, decomposition, notifications, daily timeline, GitHub/build integration.

Phase 6 - Media/Marketing

RecordingProvider, markers, MarketingCandidate, liens avec pipeline existant.

Phase 7 - Hardening

Backups, restore test, observabilite, rate limiting, security review, load/chaos tests legers. #
Checklist d'integration A/B

Avant le travail parallele

- ☐ API contract versionne.
- ☐ Event schemas versionnees.
- ☐ Auth/machine pairing defini.
- ☐ Mock fixtures partagees.
- ☐ Types Transfer et multipart definis.

Integration quotidienne

- ☐ Aucun changement incompatible non annonce.
- ☐ API change request documente si Bloc B manque un endpoint.
- ☐ Migrations DB accompagnees de schemas/API.
- ☐ Les mocks suivent le contrat reel.
- ☐ Tests contractuels executes.

Integration finale

- ☐ Deux machines reelles sur deux connexions Internet.
- ☐ Deconnexion/reconnexion testee.
- ☐ Claim conflict test reel.
- ☐ Context Package test avec Graphify + Obsidian.
- ☐ Qwen read-only memory verifie.
- ☐ Upload multipart > 5 Go teste.
- ☐ Reprise upload/download testee.
- ☐ Expiration Transfer testee.
- ☐ Backup PostgreSQL restore teste.
- ☐ TLS et rotation/revocation token verifiees.
- ☐ Dashboard, MCP et CLI coherents.